

Digital Professor: Interaction Integration

What the module provides and how it connects the full system



Jonathan Ma

October 2026

What my module provides

Shared records

- ▶ Course, session, turn, action, and evidence IDs
- ▶ Structured Student Evidence
- ▶ Confirmation and revision history
- ▶ Retrieved-source and delivery records

Integration functions

- ▶ Route data between specialized components
- ▶ Validate input before downstream use
- ▶ Prepare consistent chat, speech, math, and avatar output
- ▶ Record the full interaction for evaluation

Student Evidence: a structured, traceable record of what the student submitted, how it was interpreted, and what help preceded it.

What my module guarantees

The layer owns

- ▶ Shared identifiers and data formats
- ▶ Routing between components
- ▶ Student Evidence validation
- ▶ Confirmation, revision, and delivery history

It must always

- ▶ Preserve original student work
- ▶ Confirm meaning-changing notation
- ▶ Separate student, source, system, and tool content
- ▶ Link revisions without overwriting attempts
- ▶ Keep output consistent across channels
- ▶ Fall back to chat when another service fails

Every response must trace back to confirmed student work and supplied course sources.

What my module receives and returns

Inputs

- ▶ Verified syllabus course profile
- ▶ Retrieved lecture passages and pages
- ▶ Text, voice, handwriting, images, equations, or files
- ▶ Recognition alternatives and student corrections
- ▶ Current problem, prior attempts, and requested help
- ▶ Available delivery channels and system rules

Outputs

- ▶ Course workspace linked to indexed notes
- ▶ Validated Student Evidence
- ▶ Student Evidence with retrieved sources
- ▶ Confirmation or processing status
- ▶ Chat, speech, math, citation, and avatar package
- ▶ Complete interaction history and measurements

Every handoff includes identity, source, status, and validation. Missing information stays missing.

Core output: Student Evidence

What the student provided

- ▶ Original submission and modality
- ▶ Confirmed transcription
- ▶ Equations, steps, and answer
- ▶ Explanation and uncertainty

How it relates

- ▶ Prompt or hint that elicited it
- ▶ Attempt and revision links
- ▶ Prior evidence
- ▶ Assistance received

Where it belongs

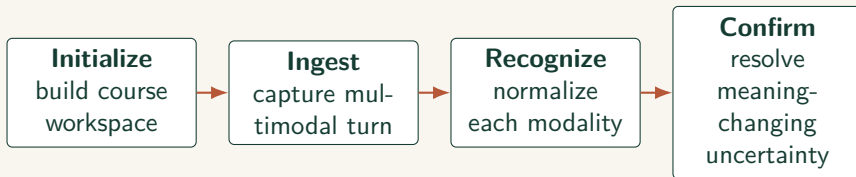
Student, course, problem, concept, turn, and time.

How it was processed

Recognition confidence, unresolved alternatives, edits, provider, latency, and cost.

The raw submission remains immutable; interpretations and revisions stay separately traceable.

What my module does: setup and input



`initialize_course_workspace`

Connect verified syllabus fields to indexed lecture notes.

`route_recognition`

Run allowed recognition tools and retain alternatives and confidence.

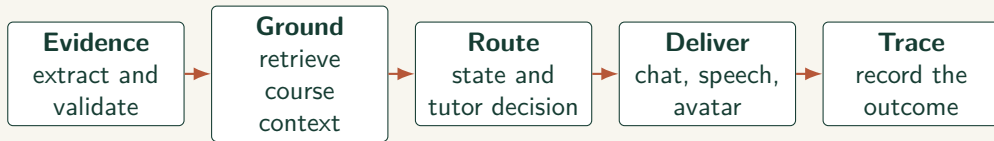
`receive_submission`

Store every input modality in one submission record.

`apply_confirmation`

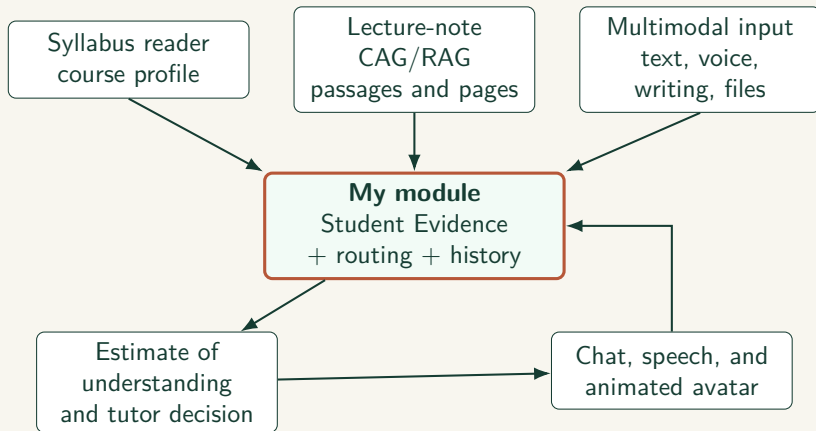
Keep the original reading and the student's correction.

What my module does: ground and deliver



- ▶ `publish_evidence`: validate the student record and notify the next component.
- ▶ `request_context`: retrieve relevant lecture passages and pages.
- ▶ `route_decision`: send evidence, sources, and the current understanding estimate.
- ▶ `prepare_delivery`: create consistent chat, math, speech, citation, and avatar content.
- ▶ `record_delivery`: record what the student received and how they responded.

How my module ties everything together



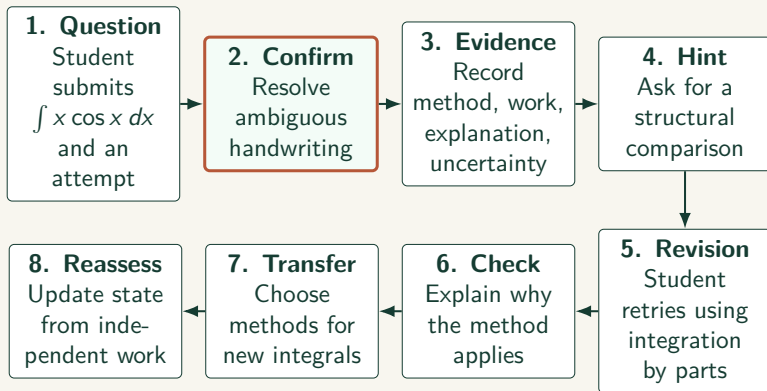
- ▶ Course and student data enter through one validated interaction record.
- ▶ One approved action drives chat, speech, mathematics, and avatar delivery.
- ▶ The response returns as the next Student Evidence event.

General tutoring workflow mapped to my module

Tutoring stage	What my module does
1. Establish course context	Build the course workspace from verified syllabus fields and indexed lecture notes.
2. Elicit student thinking	Present a question or task, then capture text, voice, handwriting, files, explanation, and uncertainty.
3. Interpret the response	Run recognition, confirm meaning-changing notation, and publish a Student Evidence event.
4. Select an intervention	Add retrieved course passages and send evidence, prior help, and source pages to the tutoring decision layer.
5. Present the intervention	Turn the approved question, hint, retry, or explanation into consistent chat, speech, math, citations, and avatar cues.
6. Reassess and continue	Link the student's revision, comprehension check, or transfer answer to the prior action and start the next cycle.



How it works in one tutoring turn



System actions presented

Question, targeted hint, retry request, explanation when needed, comprehension check, and transfer problems.

Student responses captured

Initial approach, handwriting, intermediate work, explanation, uncertainty, corrected attempt, and transfer answer.

How it ties into tutoring decisions

Integration layer sends

- ▶ Validated Student Evidence event
- ▶ Problem, concept, and requested help
- ▶ Confirmed work and uncertainty
- ▶ Prior attempts and assistance
- ▶ Retrieved passages and page IDs

Decision layer returns

- ▶ Question, hint, retry, explanation, or check
- ▶ Student-facing text and mathematics
- ▶ Amount of solution to show
- ▶ Sources and expected response
- ▶ Tool request and human-review trigger



What comes next

1. Agree on shared identifiers and data formats.
2. Define handoffs for course profiles, retrieved notes, student turns, and delivery.
3. Build one end-to-end sourced tutoring path.
4. Add handwriting confirmation and revision tracking.
5. Compare chat, speech, and avatar delivery for consistency, latency, and recovery.

First implementation target

Course setup → sourced tutoring turn → validated evidence → delivery → linked response.

Potential risks and problems

Risk	Why it could slow or block progress
No shared data contract	Components break if identifiers, statuses, or field meanings drift.
Conflicting course context	Syllabus fields and retrieved notes may be missing, stale, or contradictory.
Real-time latency	Recognition, retrieval, generation, speech, and animation may be too slow together.
Channel inconsistency	Chat, spoken math, and avatar delivery may communicate different meanings.
Confirmation burden	Too little corrupts evidence; too much interrupts tutoring.
Privacy and dependency	Voice, student work, and external providers create retention and service risks.

Immediate dependency: agree on shared data formats and handoff rules before each subsystem evolves independently.